

```
In [4]: from os.path import basename, exists
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

import nsfg

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve

        local, _ = urlretrieve(url, filename)
        print("Downloaded " + local)

download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkst
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkpl
```

```
In [5]: download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/nsfg.py")
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem
```

```
In [8]: preg = nsfg.ReadFemPreg()
preg.head()
```

```
Out[8]:
```

	caseid	pregordr	howpreg_n	howpreg_p	moscurrp	nowprgdk	pregend1	pregen
0	1	1	NaN	NaN	NaN	NaN	6.0	N
1	1	2	NaN	NaN	NaN	NaN	6.0	N
2	2	1	NaN	NaN	NaN	NaN	5.0	N
3	2	2	NaN	NaN	NaN	NaN	6.0	N
4	2	3	NaN	NaN	NaN	NaN	6.0	N

5 rows × 244 columns

Select the `birthord` column, print the value counts, and compare to results published in the [codebook](#)

```
In [10]: preg.birthord.value_counts().sort_index()
```

```
Out[10]: birthord
1.0      4413
2.0      2874
3.0      1234
4.0       421
5.0       126
6.0        50
7.0        20
8.0         7
9.0         2
10.0        1
Name: count, dtype: int64
```

We can also use `isnull` to count the number of nans.

```
In [14]: preg.birthord.isnull().sum()
```

```
Out[14]: 4445
```

Select the `prglngth` column, print the value counts, and compare to results published in the [codebook](#)

```
In [17]: preg.prglngth.value_counts().sort_index()
```

```
Out[17]: prglngth
0         15
1          9
2         78
3        151
4        412
5        181
6        543
7        175
8        409
9        594
10       137
11       202
12       170
13       446
14        29
15        39
16        44
17       253
18        17
19        34
20        18
21        37
22       147
```

```
23      12
24      31
25      15
26     117
27       8
28      38
29      23
30     198
31      29
32     122
33      50
34      60
35     357
36     329
37     457
38     609
39    4744
40    1120
41     591
42     328
43     148
44      46
45     10
46      1
47      1
48      7
50      2
```

Name: count, dtype: int64

To compute the mean of a column, you can invoke the `mean` method on a Series. For example, here is the mean birthweight in pounds:

```
In [20]: preg.totalwgt_lb.mean()
```

```
Out[20]: 7.265628457623368
```

Create a new column named `totalwgt_kg` that contains birth weight in kilograms. Compute its mean. Remember that when you create a new column, you have to use dictionary syntax, not dot notation.

```
In [23]: preg['totalwgt_kg'] = preg.totalwgt_lb / 2.2
preg.totalwgt_kg.mean()
```

```
Out[23]: 3.302558389828803
```

`nsfg.py` also provides `ReadFemResp`, which reads the female respondents file and returns a `DataFrame`:

```
In [26]: download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem")
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem")
```

```
In [28]: resp = nsfg.ReadFemResp()
```

`DataFrame` provides a method `head` that displays the first five rows:

```
In [30]: resp.head()
```

```
Out[30]:
```

	caseid	rscrinf	rdormres	rostscrn	rscreenhisp	rscreenrace	age_a	age_r	cmbir
0	2298	1	5	5	1	5.0	27	27	9
1	5012	1	5	1	5	5.0	42	42	7
2	11586	1	5	1	5	5.0	43	43	7
3	6794	5	5	4	1	5.0	15	15	10
4	616	1	5	4	1	5.0	20	20	9

5 rows × 3087 columns

Select the `age_r` column from `resp` and print the value counts. How old are the youngest and oldest respondents?

We can use the `caseid` to match up rows from `resp` and `preg`. For example, we can select the row from `resp` for `caseid` 2298 like this:

```
In [33]: resp[resp.caseid==2298]
```

```
Out[33]:
```

	caseid	rscrinf	rdormres	rostscrn	rscreenhisp	rscreenrace	age_a	age_r	cmbir
0	2298	1	5	5	1	5.0	27	27	9

1 rows × 3087 columns

And we can get the corresponding rows from `preg` like this:

```
In [35]: preg[preg.caseid==2298]
```

Out[35]:

	caseid	pregordr	howpreg_n	howpreg_p	moscurrp	nowprgdk	pregend1	pre
2610	2298	1	NaN	NaN	NaN	NaN	6.0	
2611	2298	2	NaN	NaN	NaN	NaN	6.0	
2612	2298	3	NaN	NaN	NaN	NaN	6.0	
2613	2298	4	NaN	NaN	NaN	NaN	6.0	

4 rows × 245 columns

How old is the respondent with `caseid` 1?

```
In [38]: # age for caseid 1
resp[resp.caseid==1].age_r
```

Out[38]: 1069 44
Name: age_r, dtype: int64

What are the pregnancy lengths for the respondent with `caseid` 2298?

```
In [44]: # Pregnancy klength for the respondent for caseid 22987
preg[preg.caseid==2298].prglngth
```

Out[44]: 2610 40
2611 36
2612 30
2613 40
Name: prglngth, dtype: int64

What was the birthweight of the first baby born to the respondent with `caseid` 5013?

```
In [47]: # Weight of the first baby born to the respondent of caseid 5013
# A more complete solution uses `pregordr` to select the first baby
preg[(preg.caseid==5013) & (preg.pregordr==1)].birthwgt_lb
```

Out[47]: 5516 7.0
Name: birthwgt_lb, dtype: float64

In []: